

JavaScript Arrays — Iteration, Transformation & Searching

Topics: `map()`, `filter()`, `reduce()`, `forEach()`, `find()`, `findIndex()`, `some()`, `every()`, `flat()`, `flatMap()`, `includes()`, `indexOf()`, *callbacks*, *method chaining*, *practical patterns*

1. Array Iteration কী?

Array-এর প্রতিটি element একে একে process করাকে সাধারণভাবে **array iteration** বলা হয়।

```
const numbers = [10, 20, 30, 40];

for (const number of numbers) {
  console.log(number);
}
```

JavaScript array-এর উপর কাজ করার জন্য অনেক built-in iteration methods দিয়েছে।

সবচেয়ে গুরুত্বপূর্ণ:

Method	Main Purpose	Return
<code>forEach()</code>	প্রতিটি item-এর উপর কাজ করা	<code>undefined</code>
<code>map()</code>	প্রতিটি item transform করা	নতুন array
<code>filter()</code>	কিছু item select করা	নতুন array
<code>reduce()</code>	সব item থেকে একটি result বানানো	single value / anything
<code>find()</code>	প্রথম matching item খোঁজা	element / <code>undefined</code>
<code>findIndex()</code>	প্রথম matching item's index খোঁজা	index / <code>-1</code>
<code>some()</code>	অন্তত একটি match আছে কিনা	boolean
<code>every()</code>	সব item condition মানে কিনা	boolean
<code>includes()</code>	নির্দিষ্ট value আছে কিনা	boolean
<code>indexOf()</code>	value-এর প্রথম index	number / <code>-1</code>
<code>flat()</code>	nested array flatten করা	নতুন array
<code>flatMap()</code>	map + flat করা	নতুন array

2. Callback Function

এই methods বোঝার আগে **callback function** বুঝতে হবে।

একটি function-কে অন্য function-এর argument হিসেবে পাঠালে সেটি callback function।

```
function processUser(name, callback) {  
  callback(name);  
}  
  
processUser("Shariar", function (name) {  
  console.log(name);  
});
```

Array methods-এও callback ব্যবহার হয়।

```
const numbers = [1, 2, 3];  
  
numbers.map(function (number) {  
  return number * 2;  
});
```

এখানে:

```
function (number) {  
  return number * 2;  
}
```

হলো callback।

Arrow function দিয়ে সাধারণত আরও short করে লেখা হয়:

```
const numbers = [1, 2, 3];  
  
const doubled = numbers.map(number => number * 2);  
  
console.log(doubled);
```

Output:

```
[2, 4, 6]
```

3. Callback-এর Parameters

Array iteration methods-এর callback সাধারণত তিনটি argument পেতে পারে:

```
array.map((element, index, array) => {  
  // ...  
});
```

তিনটি parameter

1. `element` → current item
2. `index` → current item's index
3. `array` → পুরো original array

Example:

```
const fruits = ["Apple", "Banana", "Mango"];  
  
fruits.map((fruit, index, array) => {  
  console.log(fruit);  
  console.log(index);  
  console.log(array);  
});
```

সাধারণত তিনটি একসাথে দরকার হয় না।

```
fruits.map((fruit, index) => {  
  return `${index + 1}. ${fruit}`;  
});
```

4. `map()` — প্রতিটি Element Transform করা

`map()` হলো JavaScript-এর সবচেয়ে গুরুত্বপূর্ণ array methods-এর একটি।

যখন তোমার দরকার:

প্রতিটি *element* থেকে নতুন কিছু তৈরি করা

তখন `map()` ব্যবহার করো।

```
const numbers = [1, 2, 3, 4];

const doubled = numbers.map(number => number * 2);

console.log(doubled);
```

Output:

```
[2, 4, 6, 8]
```

Important

`map()` original array পরিবর্তন করে না।

```
const numbers = [1, 2, 3];

const result = numbers.map(number => number * 10);

console.log(numbers);
console.log(result);
```

Output:

```
[1, 2, 3]
[10, 20, 30]
```

5. `map()` -এ Return করা জরুরি

```
const numbers = [1, 2, 3];

const result = numbers.map(number => {
  number * 2;
});

console.log(result);
```

Output:

```
[undefined, undefined, undefined]
```

কারণ `{}` ব্যবহার করলে explicit `return` দরকার।

Correct:

```
const result = numbers.map(number => {  
  return number * 2;  
});
```

অথবা:

```
const result = numbers.map(number => number * 2);
```

6. Real-world `map()` Example

ধরো API থেকে users এসেছে:

```
const users = [  
  { id: 1, name: "Shariar" },  
  { id: 2, name: "Rahim" },  
  { id: 3, name: "Karim" }  
];
```

শুধু names দরকার:

```
const names = users.map(user => user.name);  
  
console.log(names);
```

Output:

```
["Shariar", "Rahim", "Karim"]
```

7. React-এ `map()`

React-এ array থেকে UI render করতে `map()` অনেক বেশি ব্যবহার করা হয়।

```
const users = [
  { id: 1, name: "Shariar" },
  { id: 2, name: "Rahim" },
  { id: 3, name: "Karim" }
];

function UserList() {
  return (
    <ul>
      {users.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
```

এখানে `map()` প্রতিটি user থেকে একটি `<li>` তৈরি করছে।

8. `map()` এবং React `key`

React-এ list render করার সময় unique `key` দেওয়া উচিত।

```
{users.map(user => (
  <div key={user.id}>
    {user.name}
  </div>
))}
```

যদি database/API থেকে unique ID থাকে, সেটাই সাধারণত best choice।

```
key={user.id}
```

9. `filter()` — কিছু Element Select করা

`filter()` ব্যবহার করো যখন তোমার দরকার:

যে elements condition satisfy করে, শুধু সেগুলো রাখতে চাই।

Example:

```
const numbers = [1, 2, 3, 4, 5, 6];

const evenNumbers = numbers.filter(number => number % 2 === 0);

console.log(evenNumbers);
```

Output:

```
[2, 4, 6]
```

10. `filter()` কীভাবে কাজ করে?

Callback যদি:

```
true
```

return করে → element থাকবে।

Callback যদি:

```
false
```

return করে → element বাদ যাবে।

Example:

```
const numbers = [10, 20, 30, 40];

const result = numbers.filter(number => number > 20);

console.log(result);
```

Output:

```
[30, 40]
```

11. `filter()` Original Array পরিবর্তন করে না

```
const numbers = [1, 2, 3, 4];

const result = numbers.filter(number => number > 2);

console.log(numbers);
console.log(result);
```

Output:

```
[1, 2, 3, 4]
[3, 4]
```

12. Real-world `filter()` Example

```
const products = [
  { name: "Laptop", price: 80000 },
  { name: "Mouse", price: 2000 },
  { name: "Keyboard", price: 5000 }
];

const expensiveProducts = products.filter(product => {
  return product.price > 5000;
});

console.log(expensiveProducts);
```

Output:

```
[
  { name: "Laptop", price: 80000 }
]
```

13. Multiple Conditions with `filter()`

```
const users = [
  { name: "A", age: 25, active: true },
  { name: "B", age: 17, active: true },
  { name: "C", age: 30, active: false }
];

const result = users.filter(user => {
  return user.age >= 18 && user.active;
});
```

এখানে user-এর:

- age 18 বা তার বেশি হতে হবে
- active হতে হবে

14. `reduce()` — অনেকগুলো Value থেকে একটি Result

`reduce()` প্রথমে কঠিন মনে হয়।

সহজভাবে মনে রাখো:

Array-এর অনেকগুলো value নিয়ে একটি final result তৈরি করতে `reduce()` ব্যবহার করা হয়।

Example:

```
const numbers = [10, 20, 30, 40];

const total = numbers.reduce((sum, number) => {
  return sum + number;
}, 0);

console.log(total);
```

Output:

100

15. `reduce()` -এর Mental Model

এই code:

```
numbers.reduce((sum, number) => {  
  return sum + number;  
}, 0);
```

এভাবে ভাবো:

```
sum = 0  
  
0 + 10 = 10  
10 + 20 = 30  
30 + 30 = 60  
60 + 40 = 100
```

Final result:

```
100
```

16. `reduce()` Callback Parameters

```
array.reduce((accumulator, currentValue, index, array) => {  
  // ...  
}, initialValue);
```

সবচেয়ে important:

- `accumulator`
- `currentValue`
- `initialValue`

17. `accumulator` কী?

`accumulator` হলো এখন পর্যন্ত তৈরি হওয়া result।

Example:

```
const numbers = [1, 2, 3];

const result = numbers.reduce((acc, current) => {
  console.log("acc:", acc);
  console.log("current:", current);

  return acc + current;
}, 0);
```

ধারণাটা:

```
acc = 0
current = 1

acc = 1
current = 2

acc = 3
current = 3

acc = 6
```

18. `initialValue`

এই অংশ:

```
}, 0);
```

হলো initial value।

এখানে:

```
0
```

দিয়ে শুরু করছি।

আরেকটি example:

```
const numbers = [2, 3, 4];

const result = numbers.reduce((acc, number) => {
  return acc * number;
}, 1);

console.log(result);
```

Output:

24

কারণ:

```
1 × 2 = 2
2 × 3 = 6
6 × 4 = 24
```

19. `reduce()` দিয়ে Object তৈরি

`reduce()` শুধু number যোগ করার জন্য নয়।

Example:

```
const users = [
  { id: 1, name: "Shariar" },
  { id: 2, name: "Rahim" },
  { id: 3, name: "Karim" }
];

const userMap = users.reduce((acc, user) => {
  acc[user.id] = user;
  return acc;
}, {});

console.log(userMap);
```

Result:

```
{
  1: { id: 1, name: "Shariar" },
  2: { id: 2, name: "Rahim" },
  3: { id: 3, name: "Karim" }
}
```

20. `reduce()` দিয়ে Grouping

```
const products = [
  { name: "Laptop", category: "electronics" },
  { name: "Mouse", category: "electronics" },
  { name: "Shirt", category: "clothing" }
];

const grouped = products.reduce((acc, product) => {
  const category = product.category;

  if (!acc[category]) {
    acc[category] = [];
  }

  acc[category].push(product);

  return acc;
}, {});
```

Result:

```
{
  electronics: [
    { name: "Laptop", category: "electronics" },
    { name: "Mouse", category: "electronics" }
  ],
  clothing: [
    { name: "Shirt", category: "clothing" }
  ]
}
```

21. `reduce()` দিয়ে Count করা

```
const fruits = ["apple", "banana", "apple", "mango", "banana"];

const count = fruits.reduce((acc, fruit) => {
  acc[fruit] = (acc[fruit] || 0) + 1;

  return acc;
}, {});

console.log(count);
```

Output:

```
{
  apple: 2,
  banana: 2,
  mango: 1
}
```

22. `forEach()` — শুধু প্রতিটি Element-এর উপর কাজ করা

`forEach()` ব্যবহার করো যখন:

প্রতিটি element-এর জন্য কিছু কাজ করতে চাই, কিন্তু নতুন array বানানোর দরকার নেই।

```
const numbers = [1, 2, 3];

numbers.forEach(number => {
  console.log(number);
});
```

Output:

```
1
2
3
```

23. `forEach()` কোনো নতুন Array Return করে না

```
const numbers = [1, 2, 3];

const result = numbers.forEach(number => {
  return number * 2;
});

console.log(result);
```

Output:

```
undefined
```

এটাই `map()` এবং `forEach()` -এর বড় difference।

24. `map()` vs `forEach()`

`map()`

```
const result = numbers.map(number => number * 2);
```

নতুন array:

```
[2, 4, 6]
```

`forEach()`

```
numbers.forEach(number => {
  console.log(number * 2);
});
```

শুধু কাজ করবে।

সহজ Rule

New array চাই → `map()`

শুধু side effect/action → `forEach()`

25. Side Effect কী?

যে কাজ function-এর বাইরে কোনো state/data/environment পরিবর্তন করে, সেটাকে broadly side effect বলা যায়।

Example:

```
const numbers = [1, 2, 3];

numbers.forEach(number => {
  console.log(number);
});
```

`console.log()` একটি side effect।

আর:

```
let total = 0;

numbers.forEach(number => {
  total += number;
});
```

এখানে বাইরের `total` পরিবর্তন হচ্ছে।

26. `find()` — প্রথম Matching Element

`find()` ব্যবহার করো:

Condition match করা প্রথম element দরকার হলে।

```
const numbers = [10, 20, 30, 40];

const result = numbers.find(number => number > 20);

console.log(result);
```

Output:

```
30
```

40 -ও condition satisfy করে, কিন্তু `find()` প্রথম matching value-ই দেয়।

27. `find()` কিছু না পেলে

```
const numbers = [10, 20, 30];

const result = numbers.find(number => number > 100);

console.log(result);
```

Output:

```
undefined
```

28. Real-world `find()`

```
const users = [
  { id: 1, name: "Shariar" },
  { id: 2, name: "Rahim" },
  { id: 3, name: "Karim" }
];

const user = users.find(user => user.id === 2);

console.log(user);
```

Output:

```
{ id: 2, name: "Rahim" }
```

29. `filter()` vs `find()`

এটা খুব important!

`filter()`

সব matching elements দেয়।

```
const result = numbers.filter(number => number > 20);
```

Output:

```
[30, 40]
```

`find()`

শুধু প্রথম matching element দেয়।

```
const result = numbers.find(number => number > 20);
```

Output:

```
30
```

Mental Model

filter → Give me ALL matches

find → Give me the FIRST match

30. `findIndex()` — প্রথম Matching Index

`findIndex()` হলো `find()` -এর মতো, কিন্তু element-এর পরিবর্তে index দেয়।

```
const numbers = [10, 20, 30, 40];

const index = numbers.findIndex(number => number > 20);

console.log(index);
```

Output:

2

কারণ:

```
10 → index 0
20 → index 1
30 → index 2
```

31. `findIndex()` কিছু না পেলে

```
const index = numbers.findIndex(number => number > 100);

console.log(index);
```

Output:

-1

32. `find()` vs `findIndex()`

Method	Result
<code>find()</code>	element
<code>findIndex()</code>	index

Example:

```
const users = [
  { id: 1, name: "A" },
  { id: 2, name: "B" }
];

const user = users.find(user => user.id === 2);

const index = users.findIndex(user => user.id === 2);

console.log(user);
console.log(index);
```

Output:

```
{ id: 2, name: "B" }
1
```

33. `some()` — অন্তত একটি Match আছে?

`some()` check করে:

কমপক্ষে একটি element condition satisfy করে কিনা।

Result সবসময়:

```
true
```

অথবা:

```
false
```

Example:

```
const numbers = [1, 3, 5, 8];

const hasEven = numbers.some(number => number % 2 === 0);

console.log(hasEven);
```

Output:

```
true
```

কারণ `8` even।

34. `every()` — সবাই Condition মানে?

`every()` check করে:

সবগুলো *element condition satisfy* করে কিনা।

```
const numbers = [2, 4, 6, 8];

const allEven = numbers.every(number => number % 2 === 0);

console.log(allEven);
```

Output:

```
true
```

35. `some()` vs `every()`

এটা খুব সহজভাবে মনে রাখো:

```
some → অন্তত একজন
every → সবাই
```

Example:

```
const ages = [20, 25, 17, 30];

ages.some(age => age < 18);
```

Output:

```
true
```

কারণ একজনের age 18-এর নিচে।

অন্যদিকে:

```
ages.every(age => age >= 18);
```

Output:

```
false
```

কারণ সবাই 18+ নয়।

36. Real-world Validation

```
const users = [  
  { name: "A", verified: true },  
  { name: "B", verified: true },  
  { name: "C", verified: false }  
];  
  
const hasUnverifiedUser = users.some(user => !user.verified);  
  
const allVerified = users.every(user => user.verified);  
  
console.log(hasUnverifiedUser);  
console.log(allVerified);
```

Output:

```
true  
false
```

37. `includes()` — Value আছে কিনা

`includes()` check করে array-এর মধ্যে নির্দিষ্ট value আছে কিনা।

```
const fruits = ["apple", "banana", "mango"];

console.log(fruits.includes("banana"));
```

Output:

```
true
```

আর:

```
console.log(fruits.includes("orange"));
```

Output:

```
false
```

38. `includes()` Case-sensitive

```
const fruits = ["Apple", "Banana"];

console.log(fruits.includes("apple"));
```

Output:

```
false
```

কারণ:

```
"Apple" !== "apple"
```

39. `indexOf()`

`indexOf()` value-এর প্রথম index return করে।

```
const fruits = ["apple", "banana", "mango"];

console.log(fruits.indexOf("banana"));
```

Output:

```
1
```

Value না থাকলে:

```
console.log(fruits.indexOf("orange"));
```

Output:

```
-1
```

40. `includes()` vs `indexOf()`

যদি শুধু জানতে চাও value আছে কিনা:

```
array.includes(value);
```

ব্যবহার করা বেশি readable।

যদি index দরকার হয়:

```
array.indexOf(value);
```

ব্যবহার করো।

Example:

```
const numbers = [10, 20, 30];

numbers.includes(20);
```

→ `true`

```
numbers.indexOf(20);
```

→ 1

41. `indexOf()` -এর একটি Common Mistake

এই code:

```
const numbers = [10, 20, 30];

if (numbers.indexOf(10)) {
  console.log("Found");
}
```

এখানে `"Found"` print হবে না।

কারণ:

```
numbers.indexOf(10)
```

return করে:

```
0
```

আর `0` হলো falsy।

Correct:

```
if (numbers.indexOf(10) !== -1) {
  console.log("Found");
}
```

অথবা আরও clean:

```
if (numbers.includes(10)) {
  console.log("Found");
}
```

42. `flat()` — Nested Array Flatten করা

ধরো:

```
const numbers = [1, 2, [3, 4], [5, 6]];
```

এটাকে এক level flat করতে:

```
const result = numbers.flat();  
  
console.log(result);
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

Original array পরিবর্তন হয় না।

43. `flat()` Depth

Default depth হলো:

```
1
```

Example:

```
const numbers = [1, [2, [3, [4]]]];  
  
console.log(numbers.flat());
```

Output:

```
[1, 2, [3, [4]]]
```

এক level flatten হয়েছে।

44. Multiple Levels Flatten

```
const numbers = [1, [2, [3, [4]]]];

const result = numbers.flat(2);

console.log(result);
```

Output:

```
[1, 2, 3, [4]]
```

আর সব levels flatten করতে:

```
const result = numbers.flat(Infinity);
```

45. flatMap()

flatMap() basically:

```
map() + flat(1)
```

একসাথে করে।

Example:

```
const numbers = [1, 2, 3];

const result = numbers.flatMap(number => {
  return [number, number * 2];
});

console.log(result);
```

Output:

```
[1, 2, 2, 4, 3, 6]
```

46. `map()` বনাম `flatMap()`

`map()` :

```
const result = [1, 2, 3].map(number => {  
  return [number, number * 2];  
});  
  
console.log(result);
```

Output:

```
[  
  [1, 2],  
  [2, 4],  
  [3, 6]  
]
```

`flatMap()` :

```
const result = [1, 2, 3].flatMap(number => {  
  return [number, number * 2];  
});
```

Output:

```
[1, 2, 2, 4, 3, 6]
```

47. `flatMap()` কোথায় Useful?

ধরো users-এর multiple skills আছে:

```
const users = [  
  { name: "A", skills: ["React", "Node"] },  
  { name: "B", skills: ["Next.js", "TypeScript"] }  
];
```

সব skills এক array-তে চাই:

```
const skills = users.flatMap(user => user.skills);

console.log(skills);
```

Output:

```
["React", "Node", "Next.js", "TypeScript"]
```

48. `map()` + `flat()` বনাম `flatMap()`

এই দুটো প্রায় equivalent:

```
const result = users
  .map(user => user.skills)
  .flat();
```

এবং:

```
const result = users.flatMap(user => user.skills);
```

তবে `flatMap()` এক level flatten করে।

49. Method Chaining

একাধিক array method একসাথে ব্যবহার করাকে method chaining বলা হয়।

Example:

```
const numbers = [1, 2, 3, 4, 5, 6];

const result = numbers
  .filter(number => number % 2 === 0)
  .map(number => number * 10);

console.log(result);
```

Output:

```
[20, 40, 60]
```

Process:

```
[1,2,3,4,5,6]
    ↓ filter
[2,4,6]
    ↓ map
[20,40,60]
```

50. `filter()` + `map()` Real-world Example

```
const products = [
  { name: "Laptop", price: 80000, available: true },
  { name: "Mouse", price: 2000, available: false },
  { name: "Keyboard", price: 5000, available: true }
];

const productNames = products
  .filter(product => product.available)
  .map(product => product.name);

console.log(productNames);
```

Output:

```
["Laptop", "Keyboard"]
```

51. `filter()` + `map()` + `reduce()`

এবার আরও powerful example:

```
const products = [
  { name: "Laptop", price: 80000, available: true },
  { name: "Mouse", price: 2000, available: false },
  { name: "Keyboard", price: 5000, available: true }
];

const total = products
  .filter(product => product.available)
  .map(product => product.price)
  .reduce((sum, price) => sum + price, 0);

console.log(total);
```

Output:

85000

Process:

```
products
  ↓
filter available products
  ↓
[ Laptop, Keyboard ]
  ↓
map prices
  ↓
[80000, 5000]
  ↓
reduce
  ↓
85000
```

52. Method Chaining-এর সময় Order গুরুত্বপূর্ণ

এই:

```
numbers
  .filter(...)
  .map(...)
```

এবং:

```
numbers
  .map(...)
  .filter(...)
```

সবসময় একই result দেবে না।

Example:

```
const numbers = [1, 2, 3, 4];

const result1 = numbers
  .filter(number => number > 2)
  .map(number => number * 10);

console.log(result1);
```

Output:

```
[30, 40]
```

অন্যদিকে:

```
const result2 = numbers
  .map(number => number * 10)
  .filter(number => number > 2);

console.log(result2);
```

Output:

```
[10, 20, 30, 40]
```

কারণ দ্বিতীয় ক্ষেত্রে সব values `10, 20, 30, 40` হয়ে যাওয়ার পর condition check হচ্ছে।

53. `map()` VS `filter()` VS `reduce()`

Method	Main Question
<code>map()</code>	প্রতিটাকে কীভাবে transform করব?
<code>filter()</code>	কোনগুলো রাখব?
<code>reduce()</code>	সবগুলো থেকে কীভাবে একটি result বানাব?

Mental model:

```
map    → Transform
filter → Select
reduce → Accumulate
```

54. `find()` VS `filter()`

```
find    → first matching element
filter  → all matching elements
```

Example:

```
const users = [
  { id: 1, active: true },
  { id: 2, active: true },
  { id: 3, active: false }
];
```

প্রথম active user:

```
users.find(user => user.active);
```

সব active users:

```
users.filter(user => user.active);
```

55. `some()` vs `every()`

`some` → অন্তত একটি `true`?

`every` → সবগুলো `true`?

Example:

```
const scores = [80, 90, 70];

scores.some(score => score < 50);
```

→ `false`

```
scores.every(score => score >= 50);
```

→ `true`

56. `forEach()` দিয়ে `break` করা যায়?

সাধারণ `forEach()` callback থেকে `break` করা যায় না।

এটা কাজ করবে না:

```
numbers.forEach(number => {
  if (number === 3) {
    break;
  }
});
```

যদি early stop দরকার হয়, use case অনুযায়ী:

```
for...of
```

অথবা:

```
find()
some()
every()
```

ব্যবহার করা যেতে পারে।

57. `find()` কেন Useful?

`find()` condition match করলেই search শেষ করতে পারে।

Example:

```
const users = [
  { id: 1, name: "A" },
  { id: 2, name: "B" },
  { id: 3, name: "C" }
];

const user = users.find(user => user.id === 2);
```

তোমার intention খুব clear:

"ID 2-এর user-টা আমাকে দাও।"

58. `some()` কেন Useful?

যদি actual element দরকার না হয়, শুধু yes/no দরকার হয়:

```
const hasAdmin = users.some(user => user.role === "admin");
```

এটা অনেক cleaner than:

```
const admins = users.filter(user => user.role === "admin");

const hasAdmin = admins.length > 0;
```

59. `every()` দিয়ে Validation

```
const formFields = [
  { name: "email", valid: true },
  { name: "password", valid: true },
  { name: "username", valid: true }
];

const isFormValid = formFields.every(field => field.valid);

console.log(isFormValid);
```

Output:

```
true
```

60. Array Methods কি Original Array Mutate করে?

এই document-এর methods-এর মধ্যে সাধারণভাবে:

Non-mutating

```
map
filter
reduce
forEach
find
findIndex
some
every
flat
flatMap
includes
indexOf
```

এগুলো original array পরিবর্তন করে না।

তবে মনে রাখবে:

Array method non-mutating হলেও callback-এর ভিতরে তুমি object/array mutate করতে পারো।

Example:

```
const users = [
  { name: "A", active: false },
  { name: "B", active: false }
];

users.map(user => {
  user.active = true;
  return user;
});
```

এখানে `map()` নিজে array structure mutate করেনি, কিন্তু object-এর property mutate হয়েছে।

এটি React-এ বিশেষভাবে problematic হতে পারে।

61. React-এ Important Rule

React state-এর array নিয়ে কাজ করার সময় সাধারণত immutable update pattern follow করো।

Bad:

```
users.push(newUser);
setUsers(users);
```

Better:

```
setUsers(prevUsers => [...prevUsers, newUser]);
```

Filter করে remove:

```
setUsers(prevUsers =>
  prevUsers.filter(user => user.id !== userId)
);
```

Update:

```
setUsers(prevUsers =>
  prevUsers.map(user =>
    user.id === userId
      ? { ...user, active: true }
      : user
  )
);
```

এগুলো React-এর জন্য অত্যন্ত গুরুত্বপূর্ণ patterns।

62. `reduce()` কখন ব্যবহার করবে?

অনেক beginner `reduce()` everywhere ব্যবহার করতে চায়।

এটা ভালো practice না।

যদি simple transformation হয়:

```
map()
```

ব্যবহার করো।

যদি selection হয়:

```
filter()
```

ব্যবহার করো।

যদি প্রথম matching element দরকার:

```
find()
```

ব্যবহার করো।

যদি শুধু check করতে হয়:

```
some()
every()
```

ব্যবহার করো।

যখন সত্যিই accumulation দরকার:

```
reduce()
```

ব্যবহার করো।

63. Performance Mindset

ধরো:

```
const result = numbers
  .filter(number => number > 10)
  .map(number => number * 2);
```

এখানে দুইটি pass হচ্ছে:

```
filter → পুরো array process
map    → filtered array process
```

বেশিরভাগ application-এর জন্য এটা completely fine এবং readable।

শুধু premature optimization-এর জন্য সবকিছু `reduce()` -এ convert করার দরকার নেই।

Readable code সাধারণত বেশি valuable।

64. `includes()` এবং `indexOf()` Object-এর কক্ষেত্রে

Primitive values:

```
const numbers = [1, 2, 3];

numbers.includes(2);
```

→ `true`

কিন্তু object reference comparison-এর কারণে:

```
const users = [{ id: 1 }];

console.log(users.includes({ id: 1 }));
```

Output:

```
false
```

কারণ দুইটি object দেখতে একই হলেও তারা আলাদা reference।

```
{ } === { }
```

→ `false`

Object search করতে:

```
users.some(user => user.id === 1);
```

অথবা actual object পেতে:

```
users.find(user => user.id === 1);
```

65. `find()` with Optional Chaining

API data নিয়ে কাজ করার সময়:

```
const user = users.find(user => user.id === 10);

console.log(user?.name);
```

যদি user না পাওয়া যায়:

```
user === undefined
```

তবুও:

```
user?.name
```

error না দিয়ে `undefined` দেবে।

66. Empty Array-এর Behaviour

কিছু array methods empty array-তে interesting result দেয়।

```
[].some(() => true);
```

→ `false`

কারণ কোনো element-ই condition satisfy করেনি।

```
[].every(() => false);
```

→ `true`

এটি mathematical concept **vacuous truth**-এর সাথে সম্পর্কিত।

Beginner হিসেবে মনে রাখো:

```
empty.some(...) → false  
empty.every(...) → true
```

67. `reduce()` Empty Array

Initial value দিলে safe:

```
const result = [].reduce((sum, number) => {  
  return sum + number;  
}, 0);  
  
console.log(result);
```

Output:

```
0
```

কিন্তু initial value না দিলে:

```
[].reduce((sum, number) => {  
  return sum + number;  
});
```

Error হবে:

TypeError

তাই `reduce()` -এ initial value দেওয়া সাধারণত best practice।

68. `map()` থেকে Array-এর Same Length

```
const numbers = [1, 2, 3, 4];  
  
const result = numbers.map(number => number * 2);
```

Input:

4 elements

Output:

4 elements

`map()` সাধারণত one-to-one transformation করে।

69. `filter()` Array-এর Length কমাতে পারে

```
const numbers = [1, 2, 3, 4];  
  
const result = numbers.filter(number => number % 2 === 0);
```

Input:

4

Output:

2

কারণ শুধু 2 এবং 4 রাখা হয়েছে।

70. `reduce()` Array Return করতেও পারে

`reduce()` -এর result অবশ্যই number হতে হবে এমন নয়।

```
const numbers = [1, 2, 3];

const result = numbers.reduce((acc, number) => {
  acc.push(number * 2);
  return acc;
}, []);

console.log(result);
```

Output:

[2, 4, 6]

তবে এই specific কাজের জন্য `map()` অনেক বেশি readable:

```
const result = numbers.map(number => number * 2);
```

71. Method Selection Cheat Sheet

Need to transform every item?

→ `map()`

Need to keep only matching items?

→ `filter()`

Need one final accumulated result?

→ `reduce()`

Need to perform an action for every item?

→ `forEach()`

Need the first matching item?

→ `find()`

Need the first matching index?

→ `findIndex()`

Need to know if at least one matches?

→ `some()`

Need to know if all match?

→ `every()`

Need to check whether a primitive value exists?

→ `includes()`

Need the index of a value?

→ `indexOf()`

Need to flatten nested arrays?

→ `flat()`

Need map + flatten one level?

→ `flatMap()`

72. Real-world Example — E-commerce Cart

```
const cart = [
  {
    id: 1,
    name: "Laptop",
    price: 80000,
    quantity: 1
  },
  {
    id: 2,
    name: "Mouse",
    price: 2000,
    quantity: 2
  },
  {
    id: 3,
    name: "Keyboard",
    price: 5000,
    quantity: 1
  }
];
```

Product names

```
const names = cart.map(item => item.name);
```

Items above 3000

```
const expensiveItems = cart.filter(item => item.price > 3000);
```

Total price

```
const total = cart.reduce((sum, item) => {
  return sum + item.price * item.quantity;
}, 0);
```

Result:

```
89000
```

Find laptop

```
const laptop = cart.find(item => item.name === "Laptop");
```

Check whether mouse exists

```
const hasMouse = cart.some(item => item.name === "Mouse");
```

73. Real-world Example — Student Results

```
const students = [  
  { name: "A", marks: 85 },  
  { name: "B", marks: 45 },  
  { name: "C", marks: 72 },  
  { name: "D", marks: 30 }  
];
```

Passed students

```
const passed = students.filter(student => student.marks >= 40);
```

Student names

```
const names = students.map(student => student.name);
```

First student who scored 80+

```
const topStudent = students.find(student => student.marks >= 80);
```

Check whether everyone passed

```
const everyonePassed = students.every(student => student.marks >= 40);
```

Check whether anyone failed

```
const anyoneFailed = students.some(student => student.marks < 40);
```

Average marks

```
const total = students.reduce((sum, student) => {  
  return sum + student.marks;  
}, 0);  
  
const average = total / students.length;
```

74. Common Mistakes

Mistake 1 — `map()` -এ return না করা

Wrong:

```
const result = numbers.map(number => {  
  number * 2;  
});
```

Correct:

```
const result = numbers.map(number => {  
  return number * 2;  
});
```

Mistake 2 — `forEach()` থেকে নতুন Array আশা করা

Wrong:

```
const result = numbers.forEach(number => number * 2);
```

`result` হবে:

```
undefined
```

New array দরকার হলে:

```
const result = numbers.map(number => number * 2);
```

Mistake 3 — `find()` দিয়ে সব matches পাওয়া যাবে ভাবা

Wrong idea:

```
const users = users.find(user => user.active);
```

এটা শুধু first match দেবে।

সব matching users:

```
const activeUsers = users.filter(user => user.active);
```

Mistake 4 — `filter()` দিয়ে একটি item খোঁজা

যদি নিশ্চিত হও যে একটি item দরকার:

```
const user = users.find(user => user.id === id);
```

সব matching item দরকার হলে:

```
const users = list.filter(user => user.id === id);
```

Mistake 5 — `indexOf()` result সরাসরি condition হিসেবে ব্যবহার

Wrong:

```
if (array.indexOf(value)) {  
    // ...  
}
```

Correct:

```
if (array.indexOf(value) !== -1) {  
    // ...  
}
```

অথবা:

```
if (array.includes(value)) {  
    // ...  
}
```

Mistake 6 — সবকিছু `reduce()` দিয়ে করা

যেখানে:

```
map()
```

যথেষ্ট, সেখানে unnecessarily:

```
reduce()
```

ব্যবহার করলে code harder to read হতে পারে।

75. Output Prediction Practice

Question 1

```
const numbers = [1, 2, 3];

const result = numbers.map(number => number + 1);

console.log(result);
```

Output कै?

Question 2

```
const numbers = [1, 2, 3, 4, 5];

const result = numbers.filter(number => number % 2 === 1);

console.log(result);
```

Question 3

```
const numbers = [10, 20, 30];

const result = numbers.find(number => number > 15);

console.log(result);
```

Question 4

```
const numbers = [2, 4, 6];

const result = numbers.every(number => number % 2 === 0);

console.log(result);
```

Question 5

```
const numbers = [1, 2, 3];

const result = numbers.some(number => number > 5);

console.log(result);
```

Question 6

```
const numbers = [1, 2, 3, 4];

const result = numbers.reduce((sum, number) => {
  return sum + number;
}, 0);

console.log(result);
```

Question 7

```
const numbers = [1, 2, 3];

const result = numbers
  .filter(number => number > 1)
  .map(number => number * 10);

console.log(result);
```

76. Coding Practice

Challenge 1 — Double Numbers

```
const numbers = [5, 10, 15, 20];
```

`map()` ব্যবহার করে বানাও:

```
[10, 20, 30, 40]
```

Challenge 2 — Find Even Numbers

```
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];
```

Expected:

```
[2, 4, 6, 8]
```

Challenge 3 — Total Price

```
const products = [  
  { name: "Mouse", price: 2000 },  
  { name: "Keyboard", price: 5000 },  
  { name: "Monitor", price: 20000 }  
];
```

`reduce()` ব্যবহার করে total price বের করো।

Expected:

```
27000
```

Challenge 4 — Find User

```
const users = [  
  { id: 1, name: "A" },  
  { id: 2, name: "B" },  
  { id: 3, name: "C" }  
];
```

ID `3` user-কে `find()` দিয়ে বের করো।

Challenge 5 — Active Users

```
const users = [
  { name: "A", active: true },
  { name: "B", active: false },
  { name: "C", active: true }
];
```

শুধু active users বের করো।

Expected:

```
[
  { name: "A", active: true },
  { name: "C", active: true }
]
```

Challenge 6 — Check Admin

```
const users = [
  { name: "A", role: "user" },
  { name: "B", role: "user" },
  { name: "C", role: "admin" }
];
```

`some()` ব্যবহার করে check করো admin আছে কিনা।

Expected:

```
true
```

Challenge 7 — Flatten Skills

```
const users = [
  { name: "A", skills: ["React", "Node"] },
  { name: "B", skills: ["Next.js", "TypeScript"] }
];
```

`flatMap()` ব্যবহার করে:

```
["React", "Node", "Next.js", "TypeScript"]
```

বানাও।

77. Interview Questions

Beginner

1. `map()` কী?
2. `filter()` কী?
3. `reduce()` কী?
4. `forEach()` কী?
5. `map()` এবং `forEach()` -এর difference কী?
6. `find()` কী return করে?
7. `find()` কিছু না পেলে কী return করে?
8. `find()` এবং `filter()` -এর difference কী?
9. `some()` কী?
10. `every()` কী?
11. `includes()` কী?
12. `indexOf()` value না পেলে কী return করে?

Intermediate

13. `reduce()` -এর accumulator কী?
14. `reduce()` -এ initial value কেন গুরুত্বপূর্ণ?
15. `map()` callback-এর parameters কী কী?
16. `flat()` -এর default depth কত?
17. `flatMap()` কীভাবে `map()` থেকে আলাদা?
18. `some()` এবং `every()` -এর logical difference কী?
19. `findIndex()` এবং `indexOf()` -এর difference কী?
20. Array methods chain করা কী?
21. `map()` original array mutate করে কি?
22. `filter()` original array mutate করে কি?
23. `reduce()` দিয়ে object তৈরি করা যায় কীভাবে?
24. React-এ `map()` এত বেশি ব্যবহৃত হয় কেন?

78. সবচেয়ে গুরুত্বপূর্ণ Mental Model

Array methods মুখস্থ না করে এই প্রশ্নগুলো করো:

আমি কি প্রতিটি item পরিবর্তন করতে চাই?

↓
map()

আমি কি কিছু item বাদ দিতে চাই?

↓
filter()

আমি কি একটি final result বানাতে চাই?

↓
reduce()

আমি কি শুধু প্রতিটি item-এর উপর action নিতে চাই?

↓
forEach()

আমি কি প্রথম matching item চাই?

↓
find()

আমি কি প্রথম matching item's index চাই?

↓
findIndex()

আমি কি জানতে চাই অন্তত একটি match আছে?

↓
some()

আমি কি জানতে চাই সবাই condition মানে?

↓
every()

আমি কি নির্দিষ্ট value আছে কিনা জানতে চাই?

↓
includes()

আমি কি value-এর index চাই?

↓
indexOf()

আমি কি nested array flatten করতে চাই?

↓

```
flat()
```

আমি কি map করে তারপর এক level flatten করতে চাই?

↓

```
flatMap()
```

79. Quick Revision

`map()`

- প্রতিটি item transform
- নতুন array
- same number of elements

`filter()`

- condition অনুযায়ী select
- নতুন array
- elements কমতে পারে

`reduce()`

- multiple values → one accumulated result
- number/object/array/string/anything হতে পারে

`forEach()`

- প্রতিটি item-এর উপর action
- return value নেই
- undefined

`find()`

- first matching element
- না পেলে undefined

`findIndex()`

- first matching index
- না পেলে -1

`some()`

- অন্তত একটি condition satisfy?
- true / false

`every()`

- সবগুলো condition satisfy?
- true / false

`includes()`

- value exists?
- true / false

`indexOf()`

→ first index

→ না পেলো -1

`flat()`

→ nested array flatten

`flatMap()`

→ map + flat(1)

80. Golden Rules

1. *Transform* → `map()`

2. *Select* → `filter()`

3. *Accumulate* → `reduce()`

4. *Side effect/action* → `forEach()`

5. *First match* → `find()`

6. *First match index* → `findIndex()`

7. *At least one* → `some()`

8. *All* → `every()`

9. *Value exists* → `includes()`

10. *Value index* → `indexOf()`

11. *Flatten* → `flat()`

12. *Transform + flatten* → `flatMap()`

সবচেয়ে important হলো শুধু method-এর syntax মুখস্থ করা নয়—**problem** দেখেই কোন **method fit** করবে সেটা চিনতে শেখা।